

Projektuppgift

DT211G – Frontend-baserad webbutveckling

Projektuppgift

Johanna Hüll



MITTUNIVERSITETET

Avdelningen för informationssystem och -teknologi

Författare: Johanna Hüll, joni2412@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: 02, 2025

1. Introduktion

1.1 Bakgrund och problemmotivering

Dungeons & Dragons (D&D) är ett av de mest kända rollspelen inom kategorin TTRPG, vilket står för "tabletop role-playing game". Spelet skapades år 1974 av Gary Gygax och har sedan dess engagerat miljontals spelare världen över. Ett TTRPG är en form av rollspel som vanligtvis spelas vid ett bord med hjälp av papper, penna och tärningar. Spelarna skapar egna karaktärer och tar del av en berättelse som formas genom deras handlingar och beslut, ledda av en spelledare.

Dungeons & Dragons utspelar sig ofta i en fantasivärld inspirerad av genren high fantasy, liknande den som beskrivs i *Sagan om ringen*. Spelarna tar rollen som olika karaktärer, exempelvis alver, dvärgar eller människor, och samarbetar för att besegra fiender och övervinna utmaningar. Förutom att välja ras kan spelarna även välja en karaktärsklass som påverkar deras spelstil. Exempelvis kan en "fighter" använda fysiska vapen för att slåss, medan en "wizard" förlitar sig på magiska trollformler och besvärjelser. Spelmekaniken bygger på tärningsslag, där en 20-sidig tärning eller D20, används för att avgöra hur väl en karaktär lyckas med sina handlingar.

Sedan spelets lansering har det uppskattats att över 50 miljoner människor världen över har spelat Dungeons & Dragons och därmed fått möjligheten att utforska sina egna fantasivärldar och berättelser [1,2].

1.2 Övergripande syfte

En av de viktigaste delarna av Dungeons & Dragons, särskilt alla karaktärsklasser som använder sig av magi, är hanteringen av magiska trollformler eller spells. En spelare måste hålla reda på ett stort antal spells, varje med sina egna egenskaper, effekter, nivåer, skada och komponenter, vilket kan snabbt bli överväldigande. Att hitta och hålla reda på alla spellfunktioner blir ofta väldigt tidskrävande och kan bli en distraktion från själva spelets berättelse.

Syftet med detta projekt är att skapa en webbapplikation som förenklar och effektiviserar hanteringen av spells i Dungeons & Dragons, detta genom att samla information, möjliggöra sökning och integrera tärningskast för olika effekter direkt i webbläsaren för att försöka göra spelupplevelsen smidigare och mer engagerande. För att kunna ha alla de resurser man behöver på en och samma plats.

1.3 Detaljerad problemformulering

Att snabbt hitta, förstå och organisera spells under en spelsession kan vara tidskrävande och påverka spelet negativt, speciellt för nya spelare kan det vara svårt att snabbt förstå vad olika trollformler gör, hur de används och vilken skada de orsakar.

Det kan även vara svårt att behöva räkna ut skada manuellt genom att rulla tärningar och tolka trollformelnns skadevärde, särskilt vid komplicerade spells och spells på högre nivå med fler tärningar.

Applikationen riktar sig till både personer som sedan innan spelar Dungeons & Dragons eller personer som är nybörjare eller sugna på att börja spela. Genom ett sökbart bibliotek över alla spells som finns i spelet och inbyggd tärningsrullare för att förenkla spelsessionen.

2. Teori

2.1 Definition av termer och förkortningar

- API

API står för “Application Programming Interface” eller “Applikationsprogrammeringsgränssnitt” och beskriver hur olika typer av program och system kan kommunicera och interagera med varandra. Det består av ett set av regler, protokoll och funktioner som gör det möjligt för olika mjukvarukomponenter att utbyta data med varandra och utföra specifika operationer utan att behöva känna till den interna implementeringen av varandras kod.

API:er gör det möjligt för webbutvecklare att använda funktionalitet från andra webbapplikationer, tjänster eller bibliotek utan att behöva skriva om dessa funktioner själv. Genom att använda ett API kan ett system begära data eller funktioner från ett annat system och få ett svar i ett specifikt format, som exempelvis JSON [3].

- AJAX https://www.w3schools.com/js/js_ajax_intro.asp

AJAX står för Asynchronous Javascript and XML som är ett samlingsnamn för olika tekniker som tillåter Javascript på en hemsida göra anrop till en webbserver samt skriva om innehåll, utan att sidan behöver laddas om

- Mashup

En mashup är en typ av webbapplikation som sammanställer information och funktionalitet från olika oberoende källor, som exempelvis två olika API:er. Exempelvis en hemsida som använder sig av en API för att visa karta över en vald ort och en annan som visar vilket väder det är på vald plats [4].

- NodeJS

Node.js är en JavaScript miljö som gör det möjligt att köra Javascript på en server och inte bara i webbläsare. Det används ofta för att skapa webbtjänster eller realtidsapplikationer eller API:er. Det använder sig av icke-blockerande och asynkront system, vilket innebär att det kan hantera flera uppgifter samtidigt utan att vänta på att en uppgift ska bli klar innan nästa kan börja [5].

Node.js använder sig av NPM eller Node Package Manager vilket är ett pakethanteringsystem som gör det enkelt att installera och hantera externa bibliotek och

verktyg. Man kan enkelt och snabbt ladda ned och installera kod som har utvecklats av andra och som man får använda i sitt eget projekt, istället för att bygga allt från grunden själv [5].

- Parcel

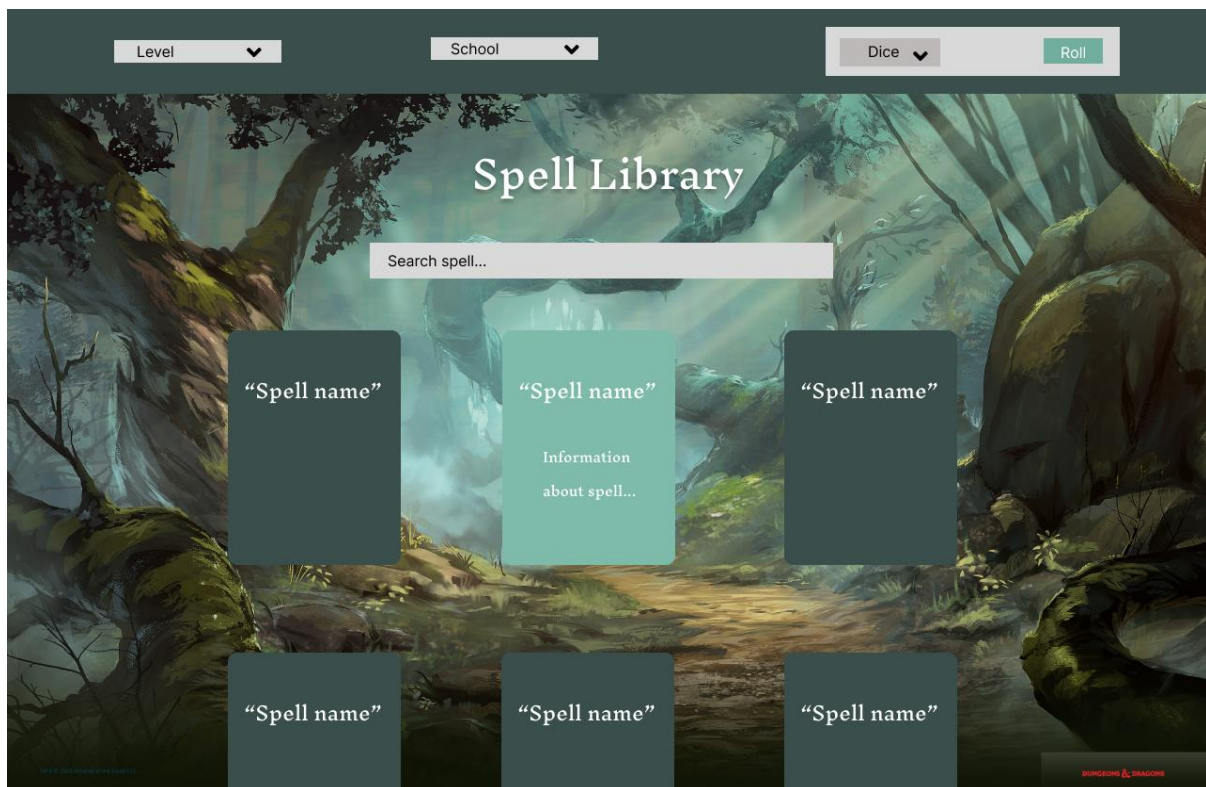
Parcel är ett modulbaserat byggsystem och bundler för webbutveckling som automatiserar processen att paketera och optimera kod för användning i en webbläsare. Parcel sköter allt från att sammanfoga filer, kompilera och minifera kod och optimera bilder [6].

.

3. Metod

- Figma

Det första jag gjorde var att göra en relativt enkel wireframe i Figma för att enklare få en övergripande bild över hur jag tänkt att webbapplikationen skulle se ut, se figur 1 nedan.



Figur 1

Jag ville ha en förhållandevis enkel design på sidan för att försöka undvika att överväldiga användaren med för många intryck samtidigt. Då spells i Dungeons and dragons oftast innehåller mycket text och information som blir svårt att ta in allt på en gång om man har alla spells samlade. Därför ville jag ha ett utseende som gör det lättare att fokusera på den relevanta information för spelaren utan att det känns för mycket.

- Grafisk profil

Jag använde mig av hemsidan coolers.co, vilket är en färgpalett generator där man kan få hjälp att skapa olika färgpaletter antingen baserat på en färg eller en bild eller redigera en befintlig palett. Med hjälp av deras färgpalett så skapade jag en passande färgpalett utifrån en bild jag valt att använda som bakgrundsbild på min sida. Denna bild laddade jag ned från [dnd.wizard.com](https://dnd.wizards.com), vilket är Dungeons & Dragons hemsidan för Wizards of the Coast som är spelföretaget som ger ut spel inom fantasy- och science fiction genren, där Dungeons & Dragons är ett av dessa spel. De bilder som finns på den sidan är fria att använda och omfattas inte av upphovsrätten.

Detta för att försöka få en enhetlig grafisk profil för sidan som är behaglig att titta på.



Figur 2

- Node.js och parcel

Under konstruktionens gång använde jag mig av Parcel som bundler tillsammans med Node.js för att kunna effektivisera och underlätta arbetet genom dess hantering av filer, paketering, optimering och uppdatering av koden under utvecklingen.

Parcel används för att hantera all Javascript, SCSS kompilering automatiskt utan att behöva göra detta manuellt. Det är också genom parcels bildkompilering som jag importerat bilderna till de olika skolorna direkt i min javascript fil.

- Kodning i visualstudio code

För själva kodningen så använde jag mig av Visual Studio Code för att skriva HTML, SCSS och Javascript.

4. Konstruktion

De API:er jag använde mig av var dels D&D 5e SRD API [7] för att hämta data med information om alla spells samt RPG Dice roller [8] för att kunna lägga till en funktion att rulla skada direkt från vald spell på hemsidan.

Jag använder mig av 2 olika AJAX funktioner för att hämta information om alla spells från API:et.

Den första funktionen `fetchSpells()` hämtar en lista över alla trollformler från API:et D&D 5e SRD, där varje spell har en index nyckel som används senare i `fetchSpellDetails(spell.index)`. Alla detaljer sparas i en array vid namn `spellsData` som är ovanför funktionerna i figur 3 nedan. När detta är klart anropas `displaysSpells(spellsData)` som är funktionen för att visa informationen om alla spells på sidan.

Andra funktionen `fetchSpellDetails(index)` hämtar alla detaljer som finns på varje spell, såsom namn, skola, beskrivning, skada eller healing per nivå samt annan information och returnerar dessa detaljer som ett objekt. För varje spell i listan så anropas denna funktion för att få detaljerna.

Jag valde att göra dessa till två olika funktioner för att lättare kunna återanvända koden, så att en användare klickar på en spell i söklistan kan man direkt hämta detaljer bara för den trollformeln, istället för att behöva inhämta all information från API:et varje gång.

```

/**
 * Hämtar alla trollformler från API:et.
 */
async function fetchSpells() {
  try {
    let response = await fetch('https://www.dnd5eapi.co/api/spells');
    let data = await response.json();
    spellsData = await Promise.all(data.results.map(async (spell) => {
      return await fetchSpellDetails(spell.index);
    }));

    displaySpells(spellsData);
  } catch (error) {
    console.error("Error fetching spells:", error);
  }
}

/**
 * Hämtar detaljerad information om en specifik trollformel.
 * @param {string} index - Index för trollformeln.
 * @returns {Promise<Object>} Detaljerad information om trollformeln.
 */
async function fetchSpellDetails(index) {
  try {
    let response = await fetch(`https://www.dnd5eapi.co/api/spells/${index}`);
    return await response.json();
  } catch (error) {
    console.error("Error fetching spell details:", error);
  }
}

```

Figur 3

Alla spells visas sedan på hemsidan genom en funktion `displaySpells(spells)` som inhämtar information om alla spells genom de tidigare funktionerna. Sedan skapas HTML-element i form av ett "spell card" för varje spell som har två sidor. Framsidan visar namnet på trollformeln samt en bild som representerar skolan. Bilderna för de olika symbolerna för skolorna är dock inte en del av API:n utan är copyright fria bilder som jag frilagt bakgrunden på och gjorde om till PNG format och importerade dem och kopplade rätt bild till rätt skola. Detta för att i min `displaySpells` funktion kunna använda bilderna och matcha dem till rätt kort. Om användaren sedan för musen över kortet så vänder det på sig och visar en baksida där man ser namnet, nivå, skola, räckvidd, komponenter och beskrivning. Samt om trollformeln har att man behöver rulla tärning så visas en dropdown och knapp för att kunna rulla tärning direkt från kortet, detta genom en händelselyssnare för att räkna ut tärningskastet när användaren klickar på knappen "roll".

`RollDamage(diceNotation)` anropas om användaren klickar på "roll" knappen. Denna funktion använder sig av det andra API:et jag använt mig av, RPG Dice Roller som räknar ut resultatet av valda tärningar, exempelvis "3d8" och returnerar ett slumpmässigt resultat.

Det finns även en separat funktion för att kunna rulla tärningar separat från korten om användaren skulle vilja eller behöva detta. Denna funktionen inhämtar också data från API:n RPG Dice Roller som är en javascript baserad tärningskast simulator som kan kasta olika typer av tärningar som används i Dungeons and dragons.

Funktionen använder sig av DOM-manipulation för att interagera med HTML-elementen och lägga till funktionalitet genom att hämta specifika element för att användas i funktionen för att hantera tärningsrullning och visning av resultat.

Sidan stylades med SCSS och använde mig av mixin för att använda färgerna i min valda färgpalett, olika fonts samt lägga till animeringar.

För att göra siden mer användarvänlig och undvika att överväldiga användaren med för mycket information på en gång så valde jag att visa alla spells i kortformat. På kortets framsida visas namnet på spellen och bilden med en symbol för vilken skola av magi det är för att göra det lite mer behagligt att titta på. När användaren för muspekaren över kortet så vänds det om och visar ytterligare information som är nödvändig för användaren att veta om varje spell.

Som visas i figur 4 nedan så görs detta i SCSS genom att använda "transform-style: preserve-3d" på kortet för att möjliggöra en tredimensionell transformering av dess fram- och baksida. Sedan "backface-visibility: hidden" gör så att baksidan av ett element döljs när det är vänt bort från användaren, vilket förhindrar att informationen på baksidan av kortet syns när det är vänt med framsidan fram.

"Transform: rotateY(180deg)" läggs till på kortets insida för att göra så att det vänds när användaren hoverar över kortet, för att simulera rörelsen av att vända på ett kort. Slutligen så lades "transition: transform 0.6s" till för att göra att vändrörelsen blir mjuk och behagligare för användaren.

Utöver detta så la jag även till "overflow-y: auto" och "::-webkit-scrollbar" för att kunna göra det möjligt att visa längre texter på kortet men att alla kort är samma storlek och behålla en enhetlig layout.

```
.card-inner {
  width: 100%;
  height: 100%;
  position: relative;
  transform-style: preserve-3d;
  transition: transform 0.6s;
}

.spell-card:hover .card-inner {
  transform: rotateY(180deg);
}
```

Figur 4

Slutligen så publicerades sidan med hjälp av Github och Netlify. Github har använts under skapandets gång för att spara ändringar och versionshantering. Netlify användes för att bygga, hosta och automatiskt publicera hemsidan, detta då man enkelt kan använda dessa tillsammans. Eftersom varje gång man pushar en ändring till github så görs en ny bygg och publiceringsprocess till Netlify och man behöver inte göra något manuellt.

5. Resultat

Webbapplikationen skapades för att hantera Dungeons & Dragons spells och erbjuder funktioner för att söka, organisera och skapa egen lista över de spells som finns i spelet. Genom att använda D&D 5e SRD API hämtas information om spells, medan RPG Dice Roller API gör det möjligt att rulla tärningar direkt i webbläsaren. Detta dels direkt från en specifik spell men även en separat.

Användargränssnittet bygger på ett kortsystem där varje spell presenteras med relevant information för spelaren. Med en framsida med namn och en baksida med ytterligare information för att inte ha överväldigande mycket text på sidan.

Tärningskast funktionen gör det möjligt att snabbt rulla skada för alla olika spells. Den implementerades genom DOM-manipulation vilket gör så att användaren kan se resultatet direkt på sidan utan att behöva använda ett annat externt verktyg.

Applikationen uppfyller de krav som gjordes genom att erbjuda en sökbar spell-databas och en funktion för att rulla tärningar. Funktionalitet testades genom att interagera med gränssnittet för att säkerställa att API-anrop fungerade korrekt.

6. Slutsats

Jag tycker att hemsidan uppfyller sitt syfte att förenkla och organisera hanteringen av spells i D&D för spelare. Den har en användarvänlig design och funktion, som att visa spells i kortformat och rulla tärningar direkt på sidan. Genom att nyttja D&D 5e SRD API och RPG Dice Roller API kunde jag effektivt integrera all nödvändiga data och funktionalitet som behövdes för applikationen.

Det finns såklart förbättringsmöjligheter.

En potentiell förbättring är att göra det möjligt att klicka på ett av spell-korten för att vända på det och att det stannar kvar vänt tills användaren klickar på det igen. Detta för att ge en mer intuitiv och kontrollerbar användarupplevelse. Sedan skulle listan för sina egna spells kunna förbättras och bli mer interaktiv.

En annan utveckling man kan göra är att förbättra tärningskasfunktionen genom att lägga till animationer och ljud av tärningar som rullar på skärmen och att det ändras beroende på vilken typ av tärning det är och hur många det är, detta för att bidra till att skapa en mer inlevelsefull upplevelse för användaren.

Sammanfattningsvis har webbapplikationen visat sig vara en användbar och effektiv lösning för spelare som söker ett lättanvänt verktyg för att hantera sina spells och snabbt rulla tärningar för att beräkna skada. Med vissa förbättringar kan applikationen bli ännu mer användarvänlig och engagerande för både nybörjare och erfarna D&D-spelare.

Källförteckning

1. Kim JH. What is Role-Playing? [Internet]. Darkshire.net; 2008 [citerad 2025 mars 20]. Tillgänglig från: <http://www.darkshire.net/~jhkim/rpg/whatis/>
2. DungeonVault. How many D&D players are there worldwide? [Internet]. 2024 [citerad 2025 mars 20]. Tillgänglig från: <https://dungeonvault.com/how-many-dnd-players-are-there-worldwide/>
3. GeeksforGeeks. What is an API? [Internet]. Tillgänglig från: <https://www.geeksforgeeks.org/what-is-an-api/>
4. Wikipedia. Mashup [Internet]. Tillgänglig från: <https://sv.wikipedia.org/wiki/Mashup>
5. Node.js. Introduction to Node.js [Internet]. Tillgänglig från: <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
6. Parcel. Parcel Documentation [Internet]. Tillgänglig från: <https://parceljs.org/>
7. D&D 5e API. [Internet]. Tillgänglig från: <https://www.dnd5eapi.co/>
8. RPG Dice Roller. Documentation [Internet]. Tillgänglig från: <https://dice-roller.github.io/documentation/>